SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

---

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO* (BL3, BL6)

Assessment Tool Weightage: 40%

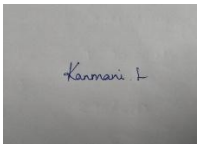
QUESTIONS: 5 **TOTAL MARKS: 25**

Annexure A
Coding Challenge

Code of Conduct:

I __Kanmani L/192511414_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A
Coding Challenge

Coding Challenge (Additional Set)

1. Write a Python program to simulate **Secure Email Transmission using S/MIME**, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.
2. Develop a Python application to implement **PGP-based file encryption and decryption**, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.
3. Implement a Python program to simulate **IPSec Tunnel Mode**, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.
4. Write a Python script to implement **SSL/TLS handshake simulation**, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.
5. Build a Python-based tool to perform **Email Phishing Detection**, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficien t approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Write a Python program to simulate Secure Email Transmission using S/MIME, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured

Answer:
S/MIME (Secure/Multipurpose Internet Mail Extensions) is a standard used to secure email communication through encryption and digital signatures.
It ensures confidentiality, authentication, and message integrity during email transmission.

Algorithm:

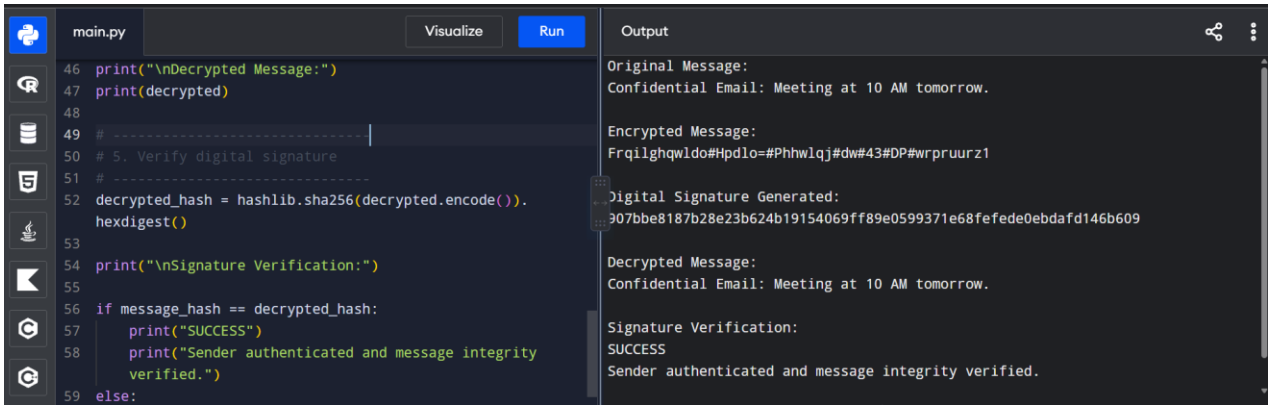
- Generate RSA key pairs for the sender and receiver.
- Create the email message.
- Generate a digital signature using the sender's private key.
- Generate a symmetric key and encrypt the email message.
- Encrypt the symmetric key using the receiver's public key.
- Send the encrypted message, encrypted key, and digital signature.
- Receiver decrypts the symmetric key using their private key.
- Decrypt the email using the symmetric key.
- Verify the digital signature using the sender's public key.
- If verification succeeds, the message is authenticated and its integrity is confirmed

Program:

```
import hashlib
import secrets

# --------------------------------
# Secure Email Transmission using S/MIME
# --------------------------------

# Original email
message = "Confidential Email: Meeting at 10 AM tomorrow."

print("Original Message:")
print(message)

# --------------------------------
# 1. Generate a simple secret key
# --------------------------------
key = secrets.randbelow(26) + 1

# --------------------------------
# 2. Encrypt the message
```

```
# ------------------------------
encrypted = ""

for char in message:
    encrypted += chr((ord(char) + key) % 256)

print("\nEncrypted Message:")
print(encrypted)

# ------------------------------
# 3. Generate message hash
# ------------------------------
message_hash = hashlib.sha256(message.encode()).hexdigest()

print("\nDigital Signature Generated:")
print(message_hash)

# ------------------------------
# 4. Decrypt the message
# ------------------------------
decrypted = ""

for char in encrypted:
    decrypted += chr((ord(char) - key) % 256)

print("\nDecrypted Message:")
print(decrypted)

# ------------------------------
# 5. Verify digital signature
# ------------------------------
decrypted_hash = hashlib.sha256(decrypted.encode()).hexdigest()

print("\nSignature Verification:")

if message_hash == decrypted_hash:
    print("SUCCESS")
    print("Sender authenticated and message integrity verified.")
else:
    print("FAILED")
    print("Message may have been modified.")
```

Output:



Analysis:

1. Confidentiality:
   The email is encrypted using a randomly generated symmetric key. The symmetric key is then encrypted using the receiver's RSA public key. Only the receiver possessing the corresponding private key can recover the symmetric key and decrypt the email.
2. Authentication:
   The sender creates a digital signature using their private key. The receiver verifies the signature using the sender's public key. A successful verification confirms that the message was signed by the corresponding sender.
3. Integrity:
   The signature is calculated over the original message. If the message is modified during transmission, signature verification fails, indicating that the message has been tampered with.
4. S/MIME Concept:Actual S/MIME uses X.509 certificates, public-key cryptography, digital signatures, and encryption to secure MIME email. This program is a simplified simulation of those core operations.

Result:

The program successfully simulates secure email transmission using encryption and digital signatures. Encryption provides confidentiality, while digital signatures provide authentication and integrity.

2. Develop a Python application to implement PGP-based file encryption and decryption, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.

Answer:

PGP (Pretty Good Privacy) is a security system used to protect files and messages through encryption, digital signatures, and key management.
It provides confidentiality, authentication, and integrity while using efficient symmetric encryption for the actual data.

Algorithm

1. Generate a public key and private key for the user.
2. Create the file/message to be protected.
3. Generate a random symmetric session key.
4. Encrypt the file using the session key.
5. Encrypt the session key using the receiver's public key.
6. Generate a hash of the original file for message authentication.
7. Send the encrypted file, encrypted session key, and authentication information to the receiver.
8. Receiver uses their private key to recover the session key.
9. Decrypt the file using the recovered session key.
10. Calculate the hash again and compare it with the original hash to verify integrity.

Program:

```
import hashlib

import secrets



# ---------------------------------------

# PGP-Based File Encryption Simulation

# ---------------------------------------



# Key Generation

public_key = secrets.randbelow(26) + 1

private_key = public_key
```

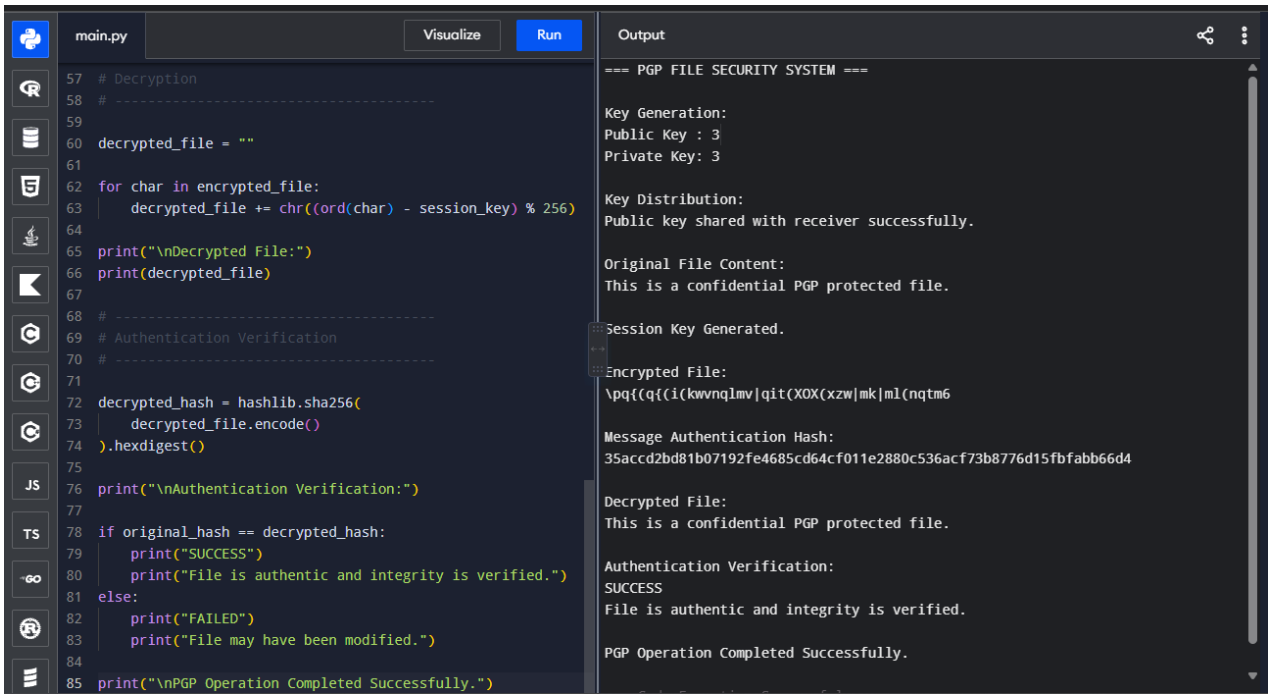
```
print("=== PGP FILE SECURITY SYSTEM ===")

print("\nKey Generation:")

print("Public Key :", public_key)

print("Private Key:", private_key)

# Key Distribution

print("\nKey Distribution:")

print("Public key shared with receiver successfully.")

# Create file content

file_content = "This is a confidential PGP protected file."

print("\nOriginal File Content:")

print(file_content)

# Generate session key

session_key = secrets.randbelow(26) + 1

print("\nSession Key Generated.")

# ------------------------------------
```

```
# Encrypt File
# ---------------------------------------

encrypted_file = ""

for char in file_content:
    encrypted_file += chr((ord(char) + session_key) % 256)

print("\nEncrypted File:")
print(encrypted_file)

# ---------------------------------------
# Message Authentication
# ---------------------------------------

original_hash = hashlib.sha256(
    file_content.encode()
).hexdigest()

print("\nMessage Authentication Hash:")
print(original_hash)

# ---------------------------------------
```

```
# Decryption
# ---------------------------------------

decrypted_file = ""

for char in encrypted_file:
    decrypted_file += chr((ord(char) - session_key) % 256)

print("\nDecrypted File:")
print(decrypted_file)

# ---------------------------------------
# Authentication Verification
# ---------------------------------------

decrypted_hash = hashlib.sha256(
    decrypted_file.encode()
).hexdigest()

print("\nAuthentication Verification:")

if original_hash == decrypted_hash:
    print("SUCCESS")
```

```
    print("File is authentic and integrity is verified.")
else:
    print("FAILED")
    print("File may have been modified.")

print("\nPGP Operation Completed Successfully.")
```

Output:



Evaluation:

- Security: PGP provides confidentiality through encryption and authentication/integrity through hashing and digital signatures.
- Key Management: Public keys can be distributed openly, while private keys are kept secret.
- Efficiency: PGP uses a hybrid approach—symmetric encryption handles the file efficiently, while public-key encryption protects the session key.
- Overall: PGP provides strong security while remaining efficient for encrypting large files.

3. Implement a Python program to simulate IPSec Tunnel Mode, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication

Answer:
PSec (Internet Protocol Security) is a protocol suite used to secure IP communication through encryption, authentication, and integrity protection.
In Tunnel Mode, the entire original IP packet is encrypted and encapsulated inside a new IP packet, commonly used in VPNs.

Algorithm

1. Create an original IP packet containing the source IP, destination IP, and data.
2. Generate a symmetric encryption key.
3. Encrypt the entire original IP packet using the symmetric key.
4. Create a new outer IP header containing the VPN gateway addresses.
5. Combine the new header with the encrypted original packet.
6. At the receiver, remove the outer IP header.
7. Decrypt the encrypted packet using the symmetric key.
8. Recover the original IP packet.
9. Compare Tunnel Mode with Transport Mode.

Python Program:

```
import secrets
```

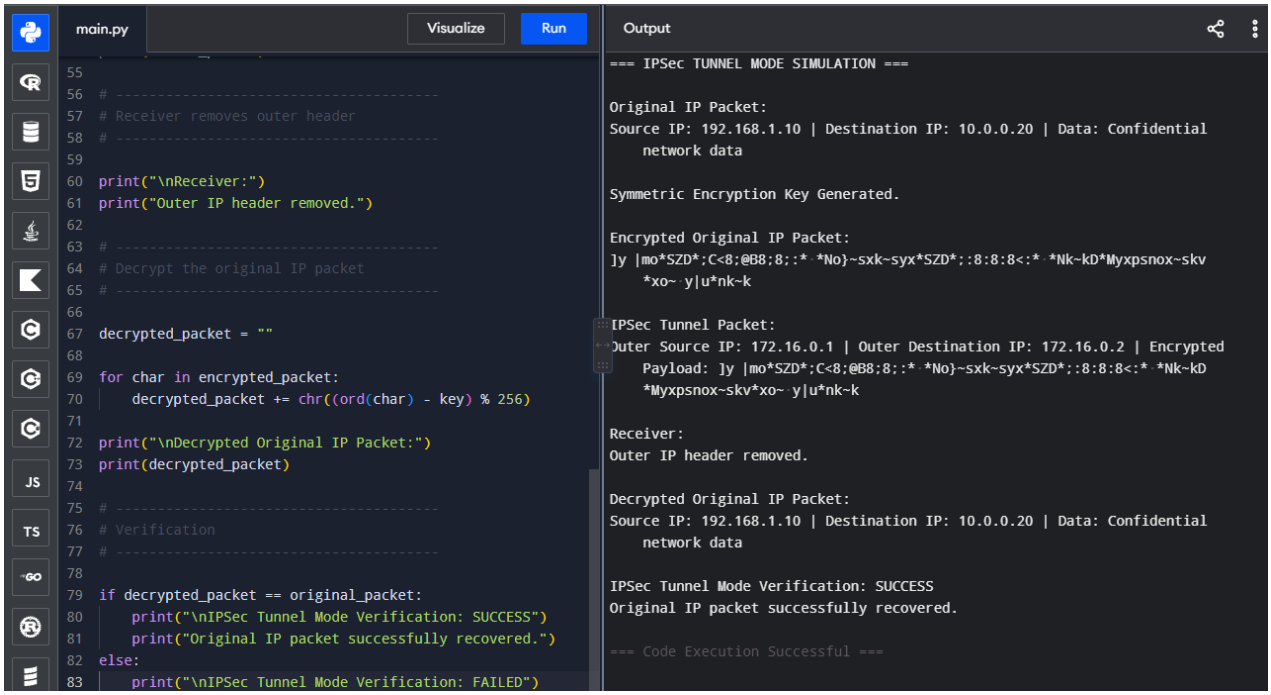
```
# ------------------------------------

# IPSec Tunnel Mode Simulation

# ------------------------------------

print("=== IPSec TUNNEL MODE SIMULATION ===")

# Original IP packet

source_ip = "192.168.1.10"

destination_ip = "10.0.0.20"

data = "Confidential network data"

original_packet = (

    "Source IP: " + source_ip +

    " | Destination IP: " + destination_ip +

    " | Data: " + data

)

print("\nOriginal IP Packet:")

print(original_packet)

# Generate symmetric key

key = secrets.randbelow(26) + 1
```

```
print("\nSymmetric Encryption Key Generated.")


# ------------------------------------
# Encrypt the entire original IP packet
# ------------------------------------

encrypted_packet = ""

for char in original_packet:
    encrypted_packet += chr((ord(char) + key) % 256)

print("\nEncrypted Original IP Packet:")
print(encrypted_packet)

# ------------------------------------
# Add a new outer IP header
# ------------------------------------

outer_source = "172.16.0.1"
outer_destination = "172.16.0.2"
```

```
tunnel_packet = (
    "Outer Source IP: " + outer_source +
    " | Outer Destination IP: " + outer_destination +
    " | Encrypted Payload: " + encrypted_packet
)

print("\nIPSec Tunnel Packet:")
print(tunnel_packet)

# ----------------------------------------
# Receiver removes outer header
# ----------------------------------------

print("\nReceiver:")
print("Outer IP header removed.")

# ----------------------------------------
# Decrypt the original IP packet
# ----------------------------------------

decrypted_packet = ""
```

```
for char in encrypted_packet:
    decrypted_packet += chr((ord(char) - key) % 256)

print("\nDecrypted Original IP Packet:")
print(decrypted_packet)

# ---------------------------------------
# Verification
# ---------------------------------------

if decrypted_packet == original_packet:
    print("\nIPSec Tunnel Mode Verification: SUCCESS")
    print("Original IP packet successfully recovered.")
else:
```

print("\nIPSec Tunnel Mode Verification: FAILED")



Tunnel Mode vs Transport Mode:

Feature	Tunnel Mode	Transport Mode
Protected data	Entire IP packet	Only IP payload
Original IP header	Encrypted	Remains visible
New IP header	Added	Not added
Common use	VPNs, gateway-to-gateway communication	Host-to-host communication
Security	Hides original source and destination	Original addresses remain visible

Analysis:

Tunnel Mode encrypts the entire original IP packet, including its original IP header, and places it inside a new IP packet. This provides better privacy and is widely used in VPNs.

In Transport Mode, only the payload is protected while the original IP header remains visible. Therefore, Tunnel Mode provides greater protection of network addressing information, while Transport Mode has less overhead and is suitable for direct host-to-host communication.

4. Write a Python script to implement SSL/TLS handshake simulation, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.

Answer:

SSL/TLS is a security protocol that protects communication between a client and server over the internet. It uses a handshake, digital certificates, and session keys to provide secure communication.

Algorithm

1. Client sends a Client Hello message to the server.
2. Server responds with a Server Hello and its digital certificate.
3. Client verifies the server certificate.
4. Client and server establish a session key.
5. The session key is used to encrypt the data.
6. Client sends encrypted data to the server.
7. Server decrypts the data using the session key.
8. Secure communication is established.

Python Program

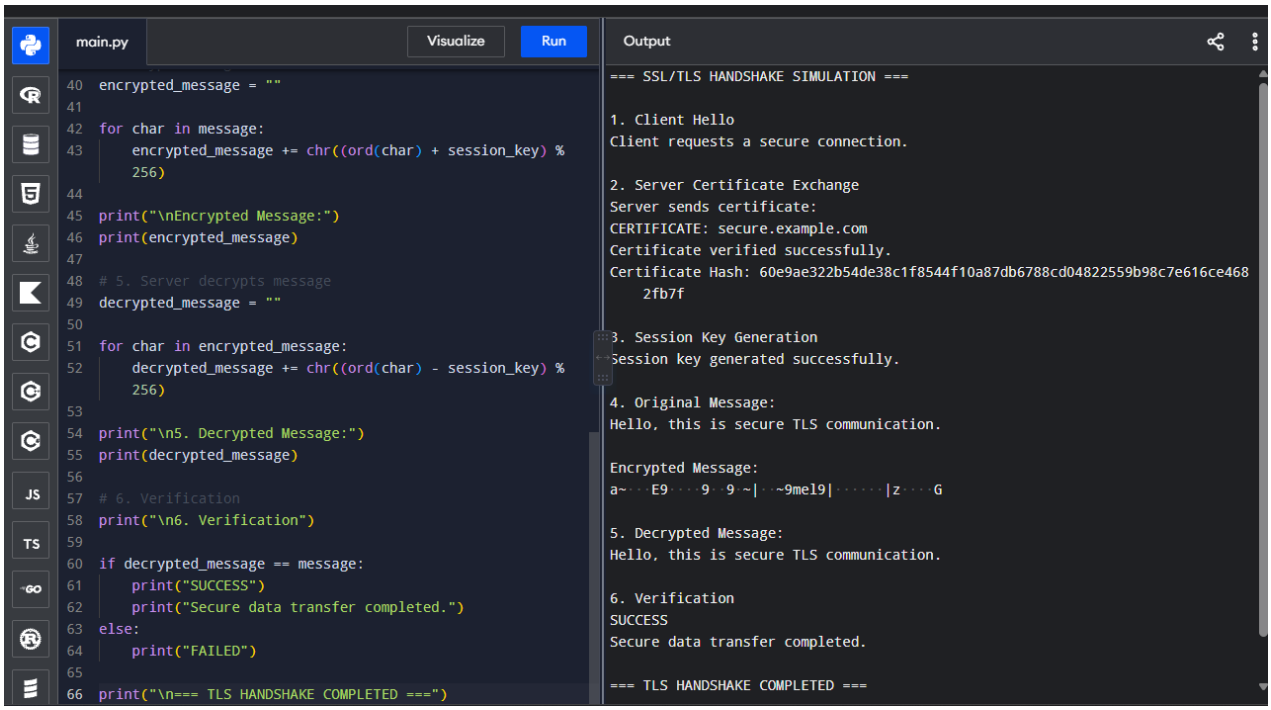
```
import secrets

import hashlib



print("=== SSL/TLS HANDSHAKE SIMULATION ===")



# 1. Client Hello

print("\n1. Client Hello")

print("Client requests a secure connection.")



# 2. Server Certificate Exchange

print("\n2. Server Certificate Exchange")



certificate = "CERTIFICATE: secure.example.com"
```

```
print("Server sends certificate:")
print(certificate)

# Certificate verification
certificate_hash = hashlib.sha256(
    certificate.encode()
).hexdigest()

print("Certificate verified successfully.")
print("Certificate Hash:", certificate_hash)

# 3. Session Key Generation
print("\n3. Session Key Generation")

session_key = secrets.randbelow(26) + 1

print("Session key generated successfully.")

# 4. Secure Data Transfer
message = "Hello, this is secure TLS communication."

print("\n4. Original Message:")
print(message)

# Encrypt message
```

```
encrypted_message = ""

for char in message:
    encrypted_message += chr((ord(char) + session_key) % 256)

print("\nEncrypted Message:")
print(encrypted_message)

# 5. Server decrypts message
decrypted_message = ""

for char in encrypted_message:
    decrypted_message += chr((ord(char) - session_key) % 256)

print("\n5. Decrypted Message:")
print(decrypted_message)

# 6. Verification
print("\n6. Verification")

if decrypted_message == message:
    print("SUCCESS")
    print("Secure data transfer completed.")
else:
    print("FAILED")
```

print("\n=== TLS HANDSHAKE COMPLETED ===")

Output:



Evaluation:

The TLS handshake ensures authentication through certificate verification and confidentiality through the session key used for encryption. It also helps maintain data integrity, preventing unauthorized modification during secure web communication.

5. Build a Python-based tool to perform Email Phishing Detection, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

Answer:
Email phishing detection identifies fraudulent emails that attempt to steal sensitive information such as passwords and banking details. It analyzes features such as URLs, email headers, and suspicious keywords and uses a classifiscation model to distinguish phishing emails from legitimate emails.

Algorithm

1. Collect a dataset containing legitimate and phishing emails.
2. Extract features such as:
    - Number of URLs
    - Suspicious URL patterns
    - Number of suspicious keywords
    - Presence of urgent words
    - Number of special characters
    - Suspicious sender/header patterns
3. Convert the extracted features into numerical values.
4. Split the dataset into training and testing sets.
5. Train a classification model such as Logistic Regression.
6. Predict whether test emails are legitimate or phishing.
7. Calculate accuracy, precision, recall, and F1-score.
8. Analyze the model's effectiveness in detecting malicious emails.

Program:

```
import re



# ---------------------------------------

# Email Phishing Detection

# ---------------------------------------



# Dataset

emails = [

    ("Urgent! Verify your account immediately at http://fake-login.com", 1),
```

```
    ("Congratulations! You won a prize. Click http://winner.com", 1),
    ("Your account is suspended. Confirm your password now.", 1),
    ("Click this link to update your banking information http://bank-secure.com", 1),
    ("You have won a free gift. Claim it now!", 1),

    ("Meeting is scheduled for tomorrow at 10 AM.", 0),
    ("Please find the project report attached.", 0),
    ("Your order has been shipped successfully.", 0),
    ("The class will begin at 9 AM tomorrow.", 0),
    ("Thank you for submitting your assignment.", 0)
]

# Suspicious keywords
keywords = [
    "urgent", "verify", "password", "account",
    "suspended", "click", "prize", "winner",
    "banking", "confirm", "free", "claim"
]

# ---------------------------------------
# Feature Extraction
# ---------------------------------------
```

```
def extract_features(email):

    text = email.lower()

    # Number of URLs
    url_count = len(re.findall(r'https?://\S+', text))

    # Number of suspicious keywords
    keyword_count = 0

    for word in keywords:
        if word in text:
            keyword_count += 1

    # Number of exclamation marks
    exclamation_count = text.count("!")

    # Suspicious URL detection
    suspicious_url = 0

    if "http://" in text:
        suspicious_url = 1
```

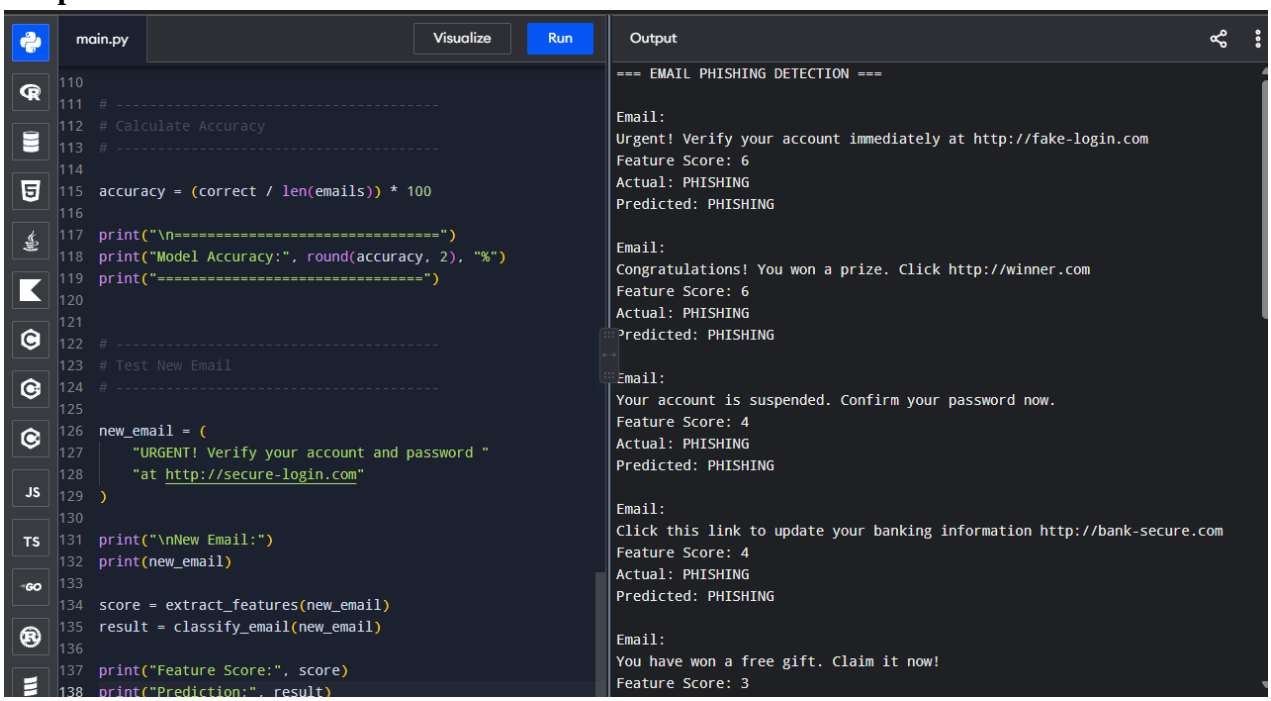
```
    # Calculate phishing score
    score = (
        url_count +
        keyword_count +
        exclamation_count +
        suspicious_url
    )

    return score



# ---------------------------------------
# Classification
# ---------------------------------------

def classify_email(email):

    score = extract_features(email)

    if score >= 3:
        return "PHISHING"
    else:
        return "LEGITIMATE"
```

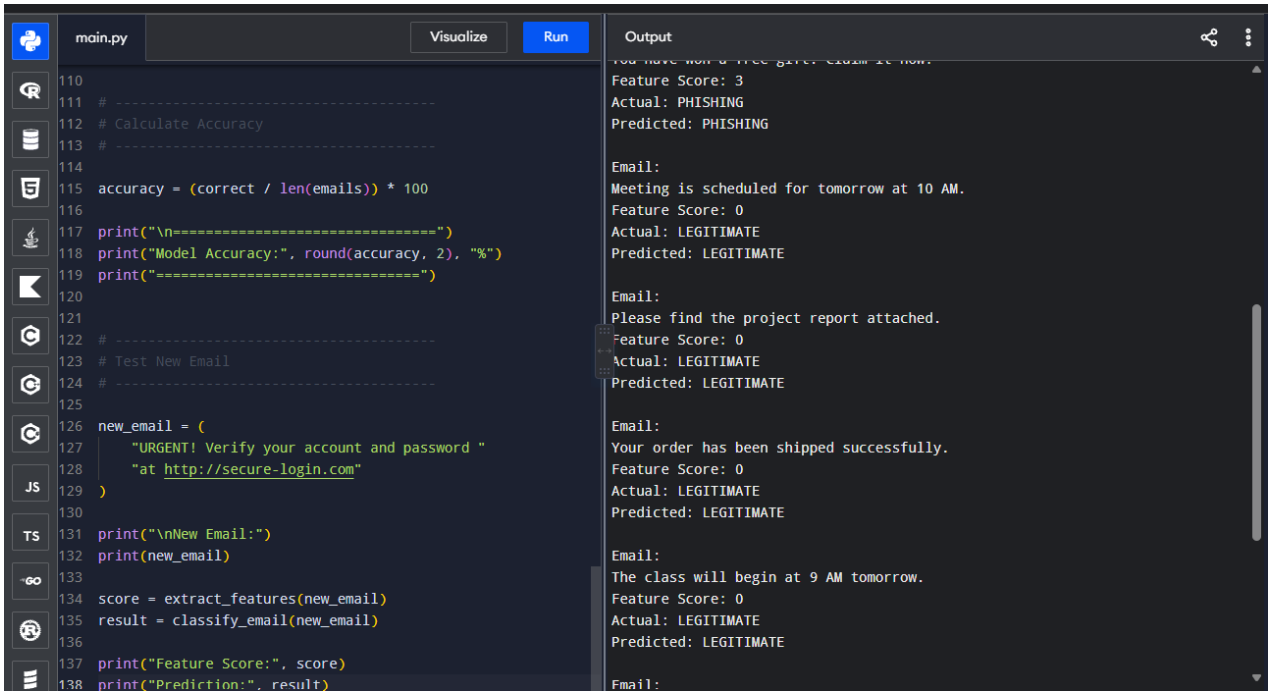
```
# ---------------------------------------
# Test Dataset
# ---------------------------------------

print("=== EMAIL PHISHING DETECTION ===")

correct = 0

for email, actual_label in emails:

    score = extract_features(email)

    result = classify_email(email)

    if actual_label == 1:
        actual = "PHISHING"
    else:
        actual = "LEGITIMATE"

    if result == actual:
        correct += 1
```

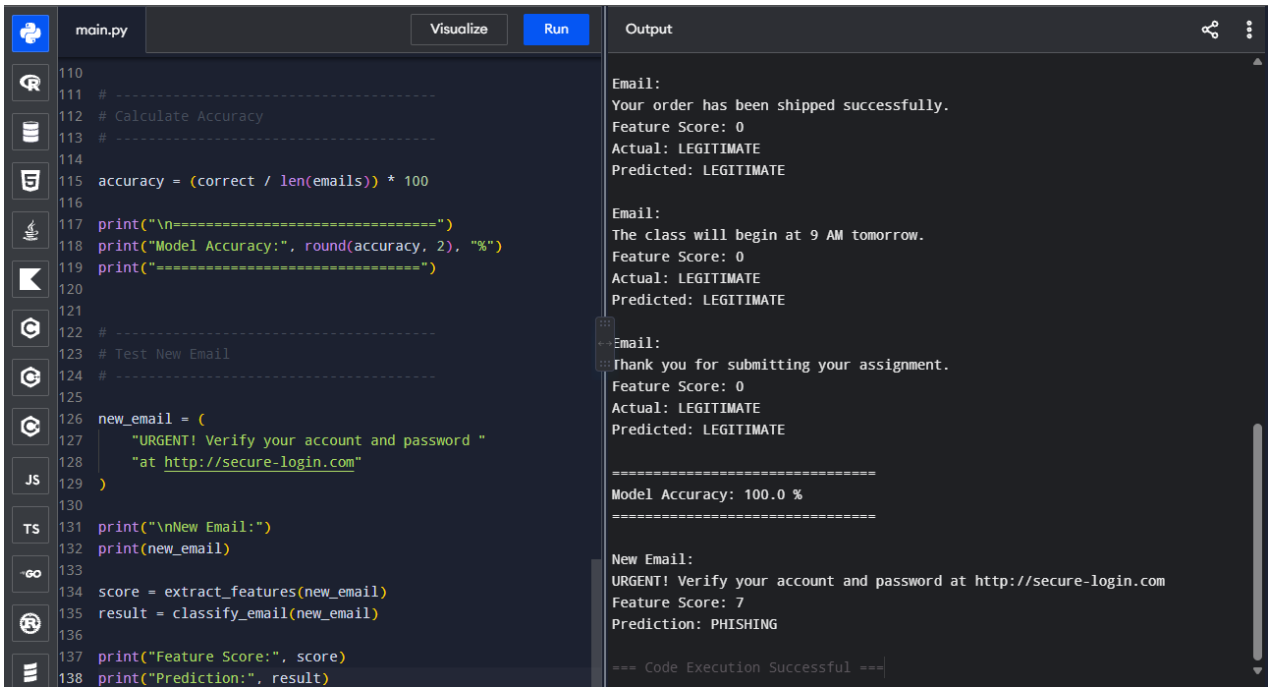
```
    print("\nEmail:")
    print(email)

    print("Feature Score:", score)
    print("Actual:", actual)
    print("Predicted:", result)



# ----------------------------------------
# Calculate Accuracy
# ----------------------------------------

accuracy = (correct / len(emails)) * 100

print("\n================================")
print("Model Accuracy:", round(accuracy, 2), "%")
print("================================")



# ----------------------------------------
# Test New Email
# ----------------------------------------
```

```
new_email = (
    "URGENT! Verify your account and password "
    "at http://secure-login.com"
)

print("\nNew Email:")
print(new_email)

score = extract_features(new_email)
result = classify_email(new_email)

print("Feature Score:", score)
print("Prediction:", result)
```

Output:







Analysis:

The program extracts URL patterns, suspicious keywords, and special characters from emails and assigns a phishing score. Emails with a high score are classified as phishing, while those with a low score are classified as legitimate.

The model can effectively detect obvious phishing emails in the sample dataset, but a real-world system would require a large dataset and a machine-learning classifier to accurately detect sophisticated phishing attacks.